GRUPPO JM

Lazzarin Andrea Silvano 899805

Gerosa Simone 894551

Sormani Francesco 894552

D'Ippolito Mattia 900030

Lietti Andrea 899951

https://github.com/simocharly03/TrackWallet



SOMMARIO

Introduzione

Architettura

Funzionalità

Activity e Fragment

Viewmodel

Repository

Service

Librerie usate

Considerazioni finali e sviluppi futuri

INTRODUZIONE

Benvenuto in TrackWallet, l'app progettata per darti il controllo completo sulle tue finanze personali. Con un'interfaccia intuitiva e funzionalità avanzate, TrackWallet ti consente di monitorare facilmente tutte le tue transazioni e organizzarle in categorie chiare e dettagliate. Con filtri personalizzabili, puoi visualizzare le tue spese e entrate per giorno, settimana o periodi di tempo specifici, aiutandoti a capire dove vanno i tuoi soldi e come gestire al meglio il tuo budget. TrackWallet ti aiuta a rimanere informato e organizzato in modo da poter prendere decisioni finanziarie intelligenti.

ARCHITETTURA

L'applicazione implementa l'architettura MVVM (Model-View-ViewModel) è un pattern utilizzato principalmente nello sviluppo di applicazioni con interfacce utente strutturate.

Componenti del pattern:

- Model (Modello)

Rappresenta i dati e la logica di business dell'applicazione.

Può interagire con database, servizi web, API.

Non ha consapevolezza della UI.

- View (Vista)

È l'interfaccia utente, ossia ciò che l'utente vede e con cui interagisce.

In WPF/XAML è composta da file XAML. In web può essere un template HTML.

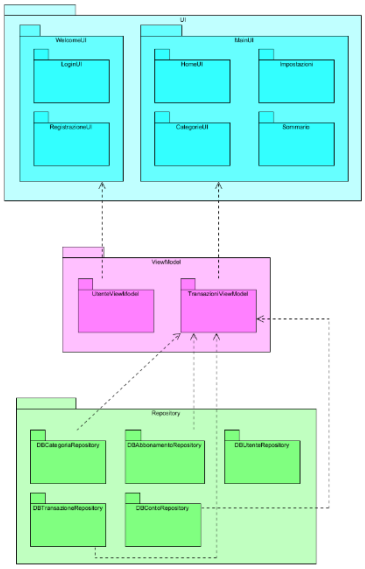
Deve essere il più possibile "stupida", ovvero non contenere logica di business.

- ViewModel

Unisce il Model e la View.

Contiene la logica di presentazione e i dati trasformati per la UI.

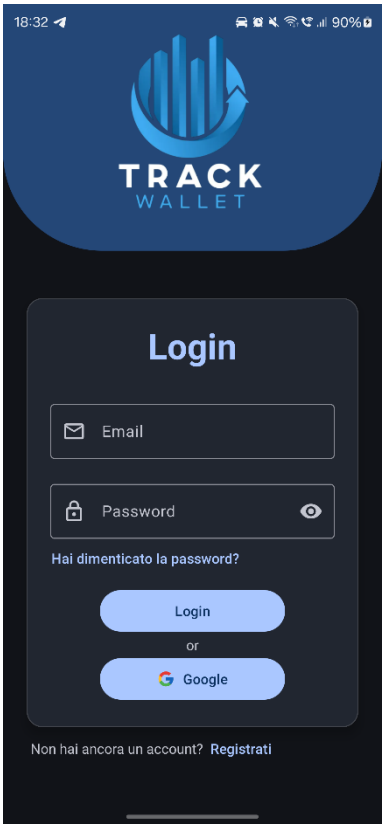
Implementa il Data Binding per collegare la View al Model.

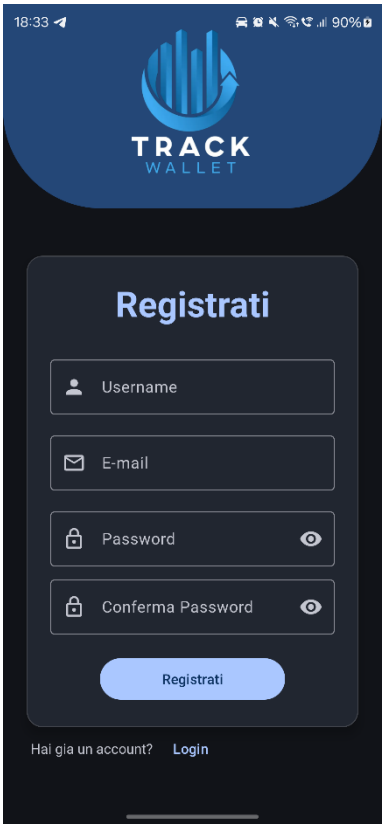


FUNZIONALITÁ

Registrazione di un profilo utente e login

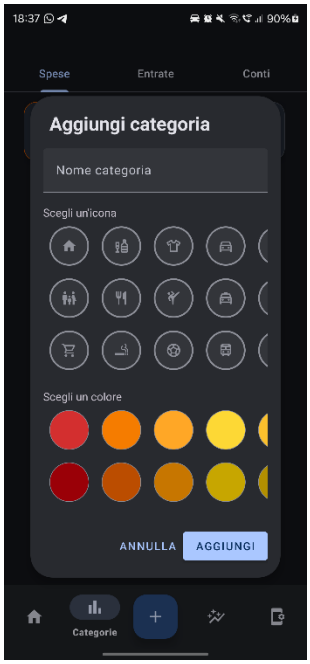
Quando viene avviata l'applicazione e necessario effettuare il login con il proprio profilo utente. Nel caso in cui l'utente non avesse creato ancora un profilo e possibile effettuare una registrazione nella quale verranno chieste username, e-mail, password e conferma password, oppure effettuare il login con Google.

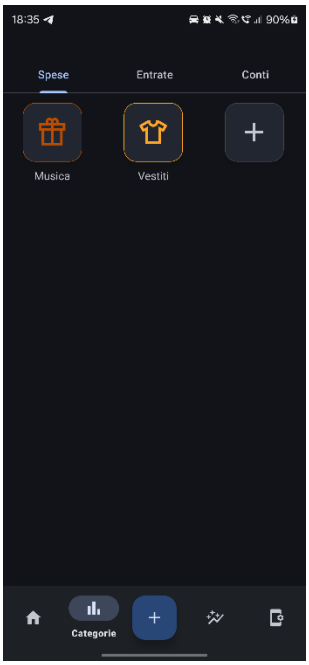




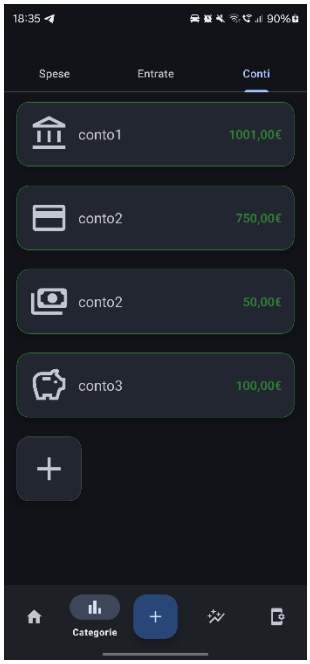
Visionare e aggiungere conti e categorie

Nella schermata categoria e possibile visionare tutte le categorie che sono state create suddivise in entrate e spese. Inoltre schiacciando su ogni categoria e possibile visionare tutte le transazioni in essa filtrandole in ordine crescente e decrescente sia per data che per importo, e anche possibile cercare all'interno della categoria le transazioni inserendo il nome di essa oppure inserendo la data in cui è stata effettuata. Sempre da questa pagina e possibile aggiungere una categoria inserendo: nome, scegliere un'icona e scegliere un colore. Infine possiamo spostarci sulla schermata dei conti dove e possibile visionarli tutti, schiacciando su un conto possiamo vedere tutte le transazioni effettuate da tale conto e per finire possiamo aggiungere un conto inserendo: nome, scegliendo un'icona e inserendo il saldo iniziale.



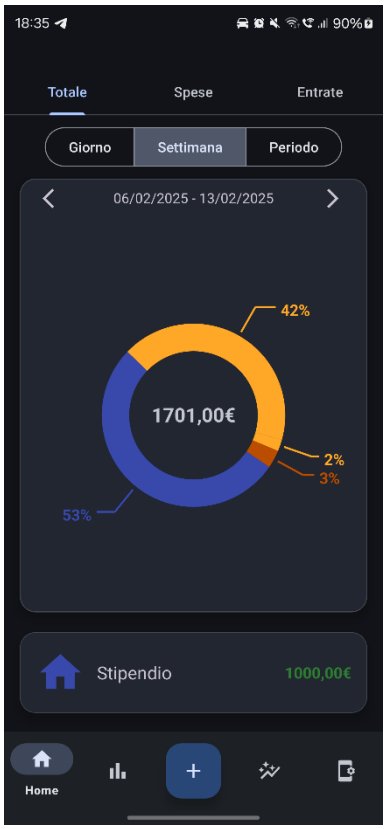






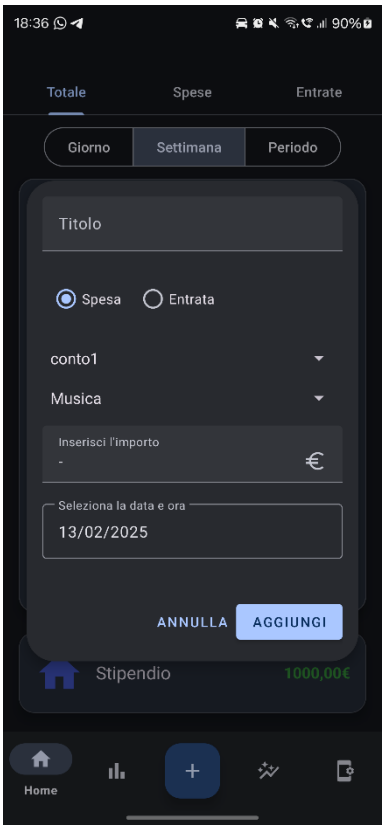
Visionare entrate spese e totale

Nella schermata home e possibile visionare mediante un grafico a torta le transazioni effettuate suddivise in categorie scegliendo se visionare solo le entrate, solo le uscite oppure il totale, inoltre l'utente può filtrare le transazioni da visionare per: giorno, settimana e periodo.



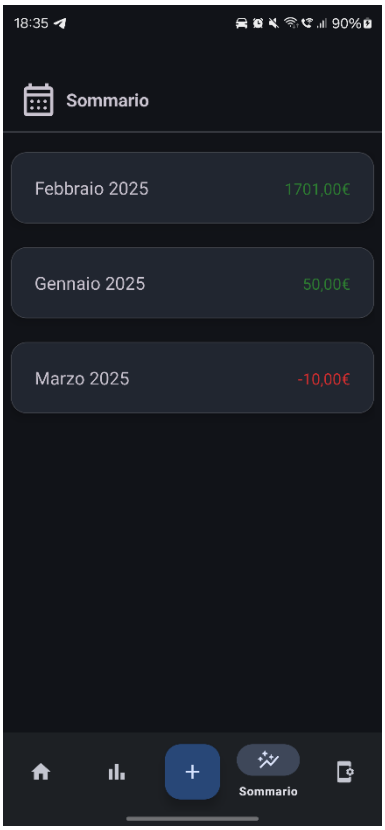
Aggiungere transazioni

Nella schermata di aggiunta e possibile aggiungere transazioni selezionando se si tratta di una spesa oppure di un'entrata, inserendo il nome del conto dalla quale l'abbiamo effettuata, inserendo il nome della categoria a cui facciamo riferimento, inserendo l'importo e infine inserendo la data di quando e stata effettuata.



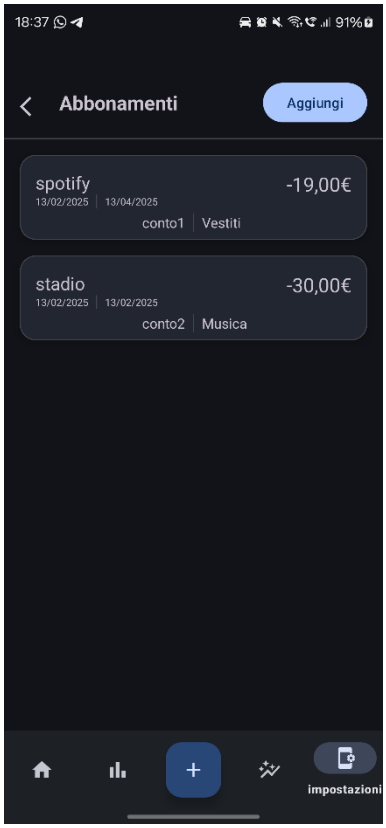
Visionare il sommario

Nella schermata sommario e possibile visionare il totale tra entrate e uscite per ogni mese, inoltre schiacciando sul mese potremmo visionare tutte le transazioni filtrandole in ordine crescente e decrescente per la data e l'importo oppure cercarle per nome.



Visionare e aggiungere abbonamenti

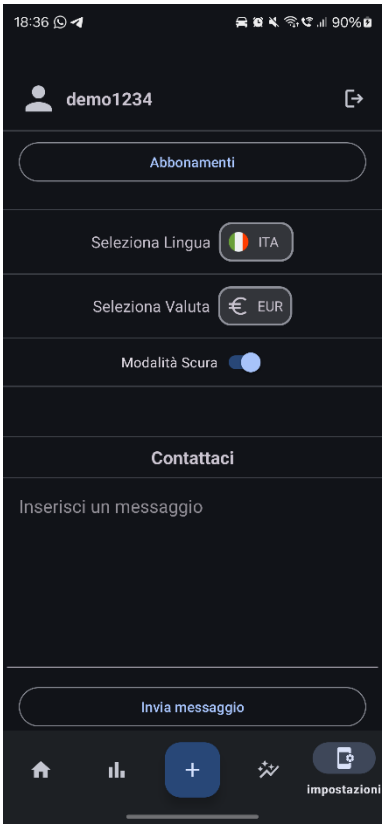
Nella schermata di impostazioni e possibile visionare gli abbonamenti che sono stati creati e aggiungerne di nuovi. Per aggiungere un abbonamento e bisogna inserire un nome, selezionare se si tratta di una spesa oppure di un'entrata, selezionare il conto a cui fare riferimento, selezionare la categoria, inserire un importo e infine inserire una data di inizio e una di fine per tale abbonamento.





Gestire preferenze e gestire account

Nella pagina impostazioni e possibile selezionare la lingua dell'applicazione mediante una API, selezionare la valuta sempre mediante l'utilizzo di una API e infine selezionare il tema per la nostra applicazione: chiaro o scuro. Invece per quanto riguarda la gestione dell'account e possibile effettuare la logout, resettare la password e infine inviare un messaggio all'assistenza.



ACTIVITY E FRAGMENT

WELCOME

LoginFragment

Gestisce il processo di login per utenti già registrati, consentendo l'accesso tramite email e password o autenticazione con Google. Verifica la validità delle credenziali inserite e, in caso di errore, mostra un messaggio all'utente. Se il login ha successo, reindirizza l'utente alla schermata principale dell'app, salvando i dati su Firebase Firestore. L'autenticazione si appoggia a FirebaseAuth e FirebaseManager per la gestione degli utenti.

RegistratiFragment

Gestisce la registrazione di un nuovo utente, verificando che username, email e password rispettino i requisiti minimi (username di almeno 6 caratteri, email valida e password di almeno 8 caratteri). Se i dati sono corretti, crea l'account su Firebase Authentication e salva le informazioni utente su Firestore. In caso di errore, segnala il problema all'utente. Se la registrazione ha successo, reindirizza l'utente alla schermata principale dell'app.

MAIN

EntrateCategorieFragment

Gestisce la visualizzazione delle categorie di entrate in una schermata a griglia. Utilizza un RecyclerView con un layout a griglia di 3 colonne per mostrare le categorie, tra cui una categoria aggiuntiva per consentire all'utente di aggiungere nuove categorie. Osserva le modifiche ai dati delle categorie tramite un DBCategoriaRepository e aggiorna l'interfaccia utente automaticamente quando i dati cambiano. L'adattatore AdapterCategoria viene utilizzato per popolare il RecyclerView con i dati delle categorie recuperati da Firebase Firestore.

EntrateHomeFragment

Gestisce la visualizzazione delle categorie di entrate in una schermata utilizzando un RecyclerView con un layout a griglia (3 colonne). Ogni categoria è visualizzata con un'icona e un nome, e c'è una colonna aggiuntiva che consente all'utente di aggiungere nuove categorie. Il codice osserva i cambiamenti nelle categorie tramite un DBCategoriaRepository e aggiorna automaticamente l'interfaccia utente ogni volta che i dati cambiano.

SpeseCategorieFragment

Gestisce la visualizzazione delle categorie di spese in una schermata utilizzando un RecyclerView con un layout a griglia (3 colonne). Ogni categoria di spesa è visualizzata con un'icona e un nome, e c'è una cella aggiuntiva che consente all'utente di aggiungere nuove categorie. Il codice osserva i cambiamenti nelle categorie tramite un DBCategoriaRepository e aggiorna automaticamente l'interfaccia utente ogni volta che i dati cambiano.

SpeseHomeFragment

Gestisce la visualizzazione delle categorie di entrate in un RecyclerView con layout a griglia (3 colonne). Ogni categoria è mostrata con un'icona e un nome. C'è una colonna aggiuntiva per permettere all'utente di aggiungere nuove categorie. Il codice osserva i cambiamenti nelle categorie tramite un DBCategoriaRepository e aggiorna automaticamente l'interfaccia utente quando i dati cambiano.

ContiFragment

Gestisce la visualizzazione dei conti in un RecyclerView con un layout a lista verticale. Ogni conto è visualizzato con un nome e un'icona. C'è anche un'ulteriore voce che permette all'utente di aggiungere nuovi conti. Il codice osserva i cambiamenti nei conti tramite un DBContoRepository e aggiorna automaticamente l'interfaccia utente ogni volta che i dati cambiano, utilizzando il DiffUtil per ottimizzare l'aggiornamento della lista.

TotaleEntrateUsciteFragment

Gestisce la visualizzazione delle spese in un fragment Android, con un grafico a torta e un elenco di categorie in un RecyclerView. L'utente può selezionare intervalli di tempo (giorno, settimana o periodo personalizzato), e i dati vengono aggiornati automaticamente tramite Firebase. Il grafico mostra le spese per categoria in percentuale, con un aggiornamento dinamico in base alla selezione.

AbbonamentiFragment

Gestisce la visualizzazione e la gestione degli abbonamenti di un utente, permettendo l'aggiunta di nuovi abbonamenti con dettagli come importo, data di inizio, data di fine, titolo e categoria. Verifica che i dati inseriti siano validi (importo corretto, date valide, campi non vuoti) e, in caso di successo, salva l'abbonamento nel database. Se la data di inizio coincide con la data corrente, crea anche una transazione associata. Per l'accesso ai dati si appoggia a DBAbbonamentoRepository, DBCategoriaRepository e DBContoRepository, che interagiscono con il database per recuperare e aggiornare le informazioni.

CategorieFragment

Gestisce la navigazione tra diverse sezioni di categorie (spese, entrate e conti) utilizzando un layout a schede (TabLayout) e un ViewPager2. Ogni scheda è associata a un fragment specifico: SpeseCategorieFragment per le spese, EntrateCategorieFragment per le entrate e ContiFragment per i conti. Il fragment utilizza un FragmentStateAdapter per gestire i fragment associati a ciascuna scheda e un TabLayoutMediator per sincronizzare le schede con il ViewPager2. Inoltre, disabilita l'effetto di ripple sulle schede per una migliore personalizzazione visiva.

HomeFragment

Gestisce la visualizzazione della schermata principale dell'app, organizzata in tre schede (TabLayout) associate a un ViewPager2. Le schede includono: Totale (visualizzato tramite TotaleEntrateUsciteFragment), Spese (gestito da SpeseHomeFragment) e Entrate (gestito da EntrateHomeFragment). Utilizza un FragmentStateAdapter per gestire i fragment associati a ciascuna scheda e un TabLayoutMediator per sincronizzare le schede con il ViewPager2. Inoltre, disabilita l'effetto di ripple sulle schede per una migliore personalizzazione visiva. Questo fragment fornisce una panoramica delle finanze dell'utente, suddivisa in totale, spese e entrate.

ImpostazioniFragment

Gestisce le impostazioni dell'app, tra cui la lingua, la valuta e il tema (scuro o chiaro). L'utente può selezionare la lingua tramite uno spinner, che salva la preferenza e applica la lingua selezionata. Allo stesso modo, la valuta può essere cambiata tramite uno spinner, con il tasso di cambio recuperato da un'API per aggiornare i prezzi nell'app. L'utente può anche attivare o disattivare il tema scuro tramite un interruttore, con il tema salvato nelle preferenze. Include anche la possibilità di contattare il supporto tramite email e fare logout.

SommarioFragment

Gestisce la visualizzazione del riepilogo mensile delle transazioni dell'utente. Recupera i dati dal database e li organizza per mese e anno, mostrando un elenco ordinato nel RecyclerView. L'utente può selezionare un mese per visualizzare i dettagli delle transazioni corrispondenti. Utilizza un ViewModel per mantenere e aggiornare i dati in tempo reale, garantendo un'interfaccia reattiva.

TransazioniCategoriaFragment

Questo fragment gestisce la visualizzazione delle transazioni filtrate per categoria. Gli utenti possono cercare transazioni specifiche, ordinarle per data o importo e visualizzarle in un RecyclerView aggiornato dinamicamente tramite DiffUtil. Utilizza un ViewModel per mantenere lo stato delle transazioni e osservare gli aggiornamenti dal database. L'interfaccia è arricchita da un Spinner per la selezione dell'ordinamento e un'app bar con navigazione.

TransazioniContoFragment

Gestisce la visualizzazione e il filtraggio delle transazioni di un conto specifico. Permette all'utente di ordinare le transazioni per data o importo e di cercarle tramite un campo di ricerca. Le transazioni vengono recuperate dal database remoto e gestite tramite un ViewModel. Inoltre, il fragment si occupa di aggiornare l'interfaccia utente con le informazioni del conto selezionato, come nome e icona, e consente la navigazione tra i vari fragment dell'app.

TransazioniMeseFragment

Gestisce la visualizzazione e la gestione delle transazioni di un utente relative a un mese e anno specifici. Permette di cercare e filtrare le transazioni per titolo, data o importo. L'utente può anche ordinare le transazioni per data (crescente o decrescente) o importo (crescente o decrescente) tramite uno spinner. Si basa su un ViewModel per osservare le transazioni filtrate e le aggiorna dinamicamente utilizzando DiffUtil per ottimizzare le performance. I dati delle transazioni vengono recuperati dal DBTransazioneRepository per l'utente corrente tramite Firebase.

TransazioniSommarioCategoria

Gestisce la visualizzazione di un sommario delle transazioni per categoria. Questo frammento utilizza un metodo di fabbrica newInstance per ricevere parametri specifici, ma al momento non implementa funzionalità specifiche. La struttura del frammento è pronta per essere estesa in futuro, e il layout è gestito dal file fragment_transazioni_sommario_categoria.

VIEWMODEL

TransazioniViewModel

La classe TransazioneViewModel gestisce la logica di business per la gestione delle transazioni finanziarie in un'applicazione. Fornisce metodi per l'aggiunta, rimozione e aggiornamento delle transazioni, nonché per il calcolo e il filtraggio dei dati, come il sommario delle transazioni per mese, categoria e tipo. I dati sono mantenuti in memoria tramite LiveData, che permette di osservare e aggiornare in tempo reale la UI dell'app. La classe si interfaccia con il repository DBCategoriaRepository per recuperare informazioni aggiuntive sulle categorie, integrando quindi funzionalità di persistenza dei dati e ottimizzazione delle operazioni asincrone.

TransazioniViewModelCategoria

La classe TransazioneViewModelCategoria gestisce la logica delle transazioni per una specifica categoria. Permette di filtrare e visualizzare le transazioni relative alla categoria selezionata, aggiungere nuove transazioni, rimuovere quelle esistenti e aggiornare i dati in tempo reale. Utilizza LiveData per monitorare e aggiornare automaticamente la UI quando vengono apportate modifiche alle transazioni o alla categoria. Inoltre, mantiene traccia dell'ID della categoria selezionata e filtra le transazioni in base ad essa, garantendo una gestione dinamica e persistenza dei dati delle transazioni per categoria.

TransazioniViewModelConto

La classe TransazioneViewModelConto gestisce la logica delle transazioni associate a un conto corrente specifico. Fornisce funzionalità per aggiungere, rimuovere e aggiornare le transazioni relative a un determinato conto. Utilizza LiveData per monitorare le modifiche alle transazioni e aggiorna automaticamente la UI quando vengono apportate modifiche. La classe consente di impostare il conto corrente, filtrare le transazioni per conto e mantenere traccia dell'ID del conto corrente. Gestisce anche l'aggiornamento in tempo reale dei dati, garantendo così una gestione efficace e persistente delle transazioni per conto.

UtenteViewModel

La classe TransazioneViewModelConto gestisce la logica delle transazioni associate a un conto corrente specifico. Permette di aggiungere, rimuovere e aggiornare le transazioni relative a un conto corrente, mantenendo il controllo costante sull'ID del conto. Utilizza LiveData per monitorare le modifiche alle transazioni, assicurando che la UI venga aggiornata automaticamente ogni volta che i dati vengono modificati. La classe fornisce funzionalità per impostare il conto corrente, filtrare le transazioni per conto e gestire la persistenza dei dati delle transazioni, garantendo anche l'aggiornamento in tempo reale delle informazioni. In questo modo, permette una gestione completa e reattiva delle transazioni legate a un conto specifico.

MODEL

Abbonamento

La classe Abbonamento rappresenta un abbonamento o pagamento periodico in un'applicazione di gestione finanziaria. Contiene informazioni come:

- **idAbbonamento**: L'identificativo unico dell'abbonamento.
- **idUtente**: L'identificativo dell'utente a cui appartiene l'abbonamento.
- **nome**: Il nome dell'abbonamento (ad esempio, "Netflix" o "Spotify").
- **conto**: Il conto associato al pagamento dell'abbonamento.
- **categoria**: La categoria dell'abbonamento (ad esempio, "Intrattenimento" o "Servizi").
- **importo**: L'importo dell'abbonamento (importo ricorrente).
- **dataInizio**: La data di inizio dell'abbonamento.
- **dataFine**: La data di fine dell'abbonamento (se presente).

La classe fornisce metodi getter e setter per ogni campo, permettendo di leggere e aggiornare i dati relativi all'abbonamento. Implementa il metodo toString() per restituire il nome dell'abbonamento, utile nelle interfacce utente.

Categoria

La classe Categoria rappresenta una categoria di spesa o entrata nell'applicazione. Contiene informazioni come:

- **idCategoria**: L'identificativo unico della categoria.
- **nome**: Il nome della categoria (ad esempio, "Spese per la casa" o "Entrate da stipendio").
- **tipo**: Indica se la categoria è una spesa (false) o un'entrata (true).
- **icona**: Un'icona associata alla categoria, utilizzata nell'interfaccia utente.
- **colore**: Un colore associato alla categoria, probabilmente per la rappresentazione visiva.
- **idUtente**: L'identificativo dell'utente a cui appartiene la categoria.

La classe fornisce metodi getter e setter per ogni campo, permettendo di leggere e aggiornare i dati relativi alla categoria. Implementa il metodo toString() per restituire il nome della categoria, utile nelle interfacce utente. Inoltre, include i metodi equals() e hashCode() per confrontare correttamente due oggetti Categoria in base al loro idCategoria, garantendo un confronto univoco tra le categorie.

Conto

La classe Conto rappresenta un conto bancario o un altro tipo di conto finanziario nell'applicazione. Contiene informazioni come:

- idConto: L'identificativo unico del conto.
- nome: Il nome del conto (ad esempio, "Conto Corrente" o "Carta di Credito").
- icona: Un'icona associata al conto, probabilmente per l'interfaccia utente.
- idUtente: L'identificativo dell'utente a cui appartiene il conto.
- saldo: Il saldo attuale del conto.

La classe fornisce metodi getter e setter per ogni campo, consentendo di leggere e aggiornare i dati del conto. Implementa il metodo toString(), che restituisce il nome del conto per visualizzarlo facilmente nelle interfacce utente. Inoltre, implementa i metodi equals() e hashCode() per garantire un confronto corretto tra due oggetti Conto in base al loro idConto. Questo consente di identificare in modo univoco e confrontare i conti.

ExchangeRateResponse

La classe ExchangeRateResponse rappresenta la risposta di un servizio che fornisce tassi di cambio tra valute. Ecco le sue caratteristiche principali:

- base_code: Il codice della valuta di base, ossia la valuta da cui si effettuano le conversioni (ad esempio, "USD" per il dollaro statunitense).
- conversion_rates: Una mappa (Map) che contiene i tassi di cambio per le valute di destinazione, dove la chiave è il codice della valuta di destinazione (ad esempio "EUR" per l'euro) e il valore è il tasso di cambio relativo.

Questa classe viene utilizzata per memorizzare la risposta di una richiesta per i tassi di cambio tra valute.

SommarioCategoria

La classe SommarioCategoria rappresenta un riepilogo per una specifica categoria. Le sue caratteristiche principali sono:

- idCategoria: L'identificativo della categoria di cui viene calcolato il totale.
- totale: Il totale delle transazioni (o valori) per quella specifica categoria.

Questa classe è utile per raccogliere e visualizzare il totale delle transazioni raggruppate per categoria.

SommarioMese

La classe SommarioMese rappresenta il riepilogo finanziario per un mese specifico. Ecco un riassunto delle sue proprietà:

- nomeMese: Il nome del mese, ad esempio "Gennaio" o "Febbraio".
- anno: L'anno di riferimento per il mese.
- bilancio: La somma totale delle transazioni per quel mese, che può essere positiva o negativa.

La classe ha anche un metodo getNumMese(), che converte il nome del mese in un numero corrispondente (es. "Gennaio" diventa 1, "Febbraio" diventa 2, ecc.).

In generale, questa classe è utile per gestire e visualizzare i riepiloghi delle transazioni per mese.

Transazione

La classe Transazione rappresenta una transazione finanziaria. Ecco un riassunto delle sue proprietà:

titolo: Il titolo o descrizione della transazione (ad esempio "Pagamento ristorante").

- idTransazione: Un identificatore univoco per la transazione.
- idUtente: L'identificativo dell'utente che ha effettuato la transazione.
- idConto: L'identificativo del conto associato alla transazione.
- idCategoria: L'identificativo della categoria associata alla transazione.
- importo: L'importo della transazione, che può essere positivo (entrata) o negativo (spesa).
- data: La data in cui la transazione è stata effettuata, rappresentata come un Timestamp.

Inoltre, la classe implementa i metodi equals() e hashCode() per confrontare due transazioni in base al loro idTransazione.

Questa classe è utilizzata per gestire e registrare le transazioni finanziarie, con informazioni cruciali come l'importo, la categoria, e il conto.

Utente

La classe Utente rappresenta un utente dell'applicazione. Ecco un riassunto delle sue proprietà:

- idUtente: Un identificatore univoco per l'utente.
- username: Il nome utente utilizzato per l'autenticazione o l'interazione nell'applicazione.
- mail: L'indirizzo email associato all'utente.

Questa classe serve a memorizzare le informazioni di un utente, come il suo ID, nome utente e indirizzo email.

REPOSITORY

- **DBAbbonamentoRepository**
- **DBCategoriaRepository**
- **DBContoRepository**
- **DBTransazioneRepository**
- **DBUtenteRepository**

Contengono tutti i metodi necessari per gestire le informazioni nel database Firestore, supportando l'approccio online/offline e sincronizzando i dati in modo efficiente.

SERVICE

Google Sign-In

Mette a disposizione le chiamate API per consentire agli utenti di accedere facilmente all'applicazione utilizzando il proprio account Google.



google-services.jso
n

(da inserire in ..\TrackWallet\app)

ExchangeRate

Mette a disposizione le chiamate API per convertire una valuta in un'altra utilizzando il tasso di cambio in tempo reale.

CURRENCY_API_KEY=c0ae6d7594c95130cf8ca7fe

(da inserire in local_properties)

LIBRERIE UTILIZZATE

Firebase

- Firebase Authentication: offre un servizio di autenticazione per gli utenti
- Firebase Firestore: permette di salvare i dati.
- Firebase Functions: permette di effettuare funzioni Javascript a livello di database (Abbonamenti).

Retrofit

Permette l'utilizzo delle richieste api.

Commons Validator

Svolge i controlli dei campi inseriti in generale.

MP Android Chart

Permette di creare grafici per i dati.

Material

Fornisce i componenti grafici dello standard Material 3.

Recycler View

Fornisce l'utilizzo delle liste dinamiche.

CONSIDERAZIONI FINALI E SVILUPPI FUTURI

L'applicazione soddisfa tutti i prerequisiti prefissati, offrendo un funzionamento efficiente e fluido. Sebbene sia già ben strutturata, ci sono margini di miglioramento, come l'aggiunta di controlli e ottimizzazioni, per ridurre al minimo l'errore umano.

Per migliorare l'esperienza utente, sarebbe utile implementare caricamenti fluidi, come gli effetti Shimmer.

Inoltre, per rendere l'applicazione più automatizzata, sarebbe possibile collegare conti bancari reali, permettendo l'importazione diretta delle transazioni, senza necessità di inserimento manuale.

Tuttavia, in tal caso, sarebbe fondamentale rafforzare la sicurezza del sistema, anche attraverso l'uso della crittografia.